Tutorial on Protein-Peptide Molecular Docking Using LightDock and MolAICal---Also Applicable to Protein-Protein Docking

Qifeng Bai
Email: molaical@yeah.net
Homepage: https://molaical.github.io or https://molaical.gitlab.io
School of Basic Medical Sciences
Lanzhou University
Lanzhou, Gansu 730000, P. R. China

1. Introduction

MolAICal can use the open-source LightDock[1] software (https://lightdock.org) and further enhances the docking accuracy of biomacromolecules through MM/GBSA or MM/PBSA methods. LightDock is an open-source (GPL-3.0 license), **docking framework** for **protein-protein**, **protein-peptide**, **protein-RNA,** and **protein-DNA** complexes. It handles flexible and challenging cases (e.g., transient interactions, low-affinity complexes, or systems requiring backbone/side-chain flexibility) **particularly well and serves as an extensible platform for testing new scoring functions, restraints**, or **optimization strategies.** LightDock adapts the Glowworm Swarm Optimization (GSO) algorithm[2], a bio-inspired swarm-intelligence method originally developed for multimodal function optimization. This tutorial uses the glucagon-like peptide-1 receptor (GLP-1R) and exendin-4, the first FDA-approved GLP-1R agonist, as demonstration examples (PDB ID: 7LLL.pdb).

This tutorial is only applicable for completion on the Linux operating system! Only some software can be completed on Windows. Users can transfer files between Linux and Windows via SSH shell or other tools to better observe the running results.

2. Materials

2.1. Software requirement

1) MolAICal: https://molaical.github.io
2) NAMD (**CPU version**): https://www.ks.uiuc.edu/Research/namd/
(**Notice:** This tutorial can be run by **NAMD 2.x, 3.x, or a higher version.** For example, the command "**namd3**" substitutes for the command "**namd2**" in this tutorial **if you use NAMD 3.x version**. For a higher version of NAMD, users can use a similar replacement way as the previous example.)
3) PyMol: https://github.com/cgohlke/pymol-open-source-wheels
4) VMD: https://www.ks.uiuc.edu/Research/vmd

2.2. Example files

1) All the necessary tutorial files are downloaded from:
https://github.com/MolAICal/tutorials/tree/master/022-protein_pp_docking

3. Procedure

3.1. Preparing for molecules

To address the library dependency issues in Linux, **the Linux version of MolAICal (Not for Windows version)** adopts a container-based approach. If **external programs need** to be invoked, it is **recommended** to install them **within** the MolAICal **container**. If **no external** programs are

required, this step can be **ignored**. This tutorial requires the external programs NAMD and VMD, and the steps are as follows:

a) First of all, copy the files to the MolAICal container.

```
****** 1.  Go to the container file system, it will enter the "/root" ******
#> molaical.exe -eset shell in

****** 2.  The first part of 'cp' is from the local machine, and the second part is in the container; ******
****** The VMD and NAMD packages can be moved into the container by the command below ******
#> cp /home/user/<local machine file> /root/soft

****** 3.  Exit container file system ******
#> exit
```

b) Secondly, enter the container virtual environment of MolAICal:

```
#> molaical.exe -eset sys run molaical
```

Note: 'molaical' is the container name.

c) Installing software in the container virtual environment of MolAICal is the same as installing it on the host local machine. Here, we take the installation of VMD and NAMD as examples:

1) For NAMD:

Extract the NAMD files, assuming the extracted folder is named namdcpu. Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter appears to be ineffective for reproducibility in the CUDA version of NAMD.

2) For VMD:

Following the steps below:

✧ Extract the VMD files:

```
#> tar -xzvf vmd-xxx.tar.gz
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system.

✧ Modify the install path in a file named "**configure**" in the decompressed folder of VMD:

```
The default: $install_bin_dir="/usr/local/bin"; modify it to →
$install_bin_dir="/root/soft/vmd193/bin";

The default: $install_library_dir="/usr/local/lib/$install_name"; modify it to →
$install_library_dir="/root/soft/vmd193/lib/$install_name";
```

✧ Install VMD:

```
#> cd vmd-xx
#> ./configure LINUXAMD64
#> cd src
#> make install
```

Note: Please run **./configure**, and select the right types for the used computers. Here, it is "LINUXAMD64".

✧ Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
```

So far, all the installations and configurations of NAMD and VMD are complete in the MolAICal container.

To prevent the loss of installed programs (e.g., VMD and NAMD) when rebuilding the MolAICal image, refer to step 3 in **Appendix 1**.

✧ **Remember to use the "exit" command to quit the MolAICal virtual environment and return to the local computer for computation (primarily to avoid file-copying steps; execution within the container is also possible but requires copying data from the local computer to the MolAICal container).**

```
#> exit
```

Please note that the configuration of MolAICal differs between the Windows and Linux versions: the Linux version utilizes the udocker container approach and requires setup within the container, whereas the Windows version does not.

Assuming users have configured the internal commands in MolAICal for VMD and NAMD using the following instructions:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter appears to be ineffective for reproducibility in the CUDA version of NAMD.

Open the tutorial material folder "022-protein_pp_docking":
#> cd 022-protein_pp_docking

The files "R_P_complex.pdb" (chain name is "P" and "R"), "R_protein.pdb" (chain name is "R"), and "P_peptide.pdb" (chain name is "P") are extracted from the complex of the glucagon-like peptide-1 receptor (GLP-1R) and exendin-4, the first FDA-approved GLP-1R agonist (PDB ID: 7LLL.pdb) by PyMol as follows:

Load PDB file 7LLL.pdb into PyMol, and use the steps of **Figure 1** to save "R_P_complex.pdb", "R_protein.pdb", and "P_peptide.pdb", respectively.

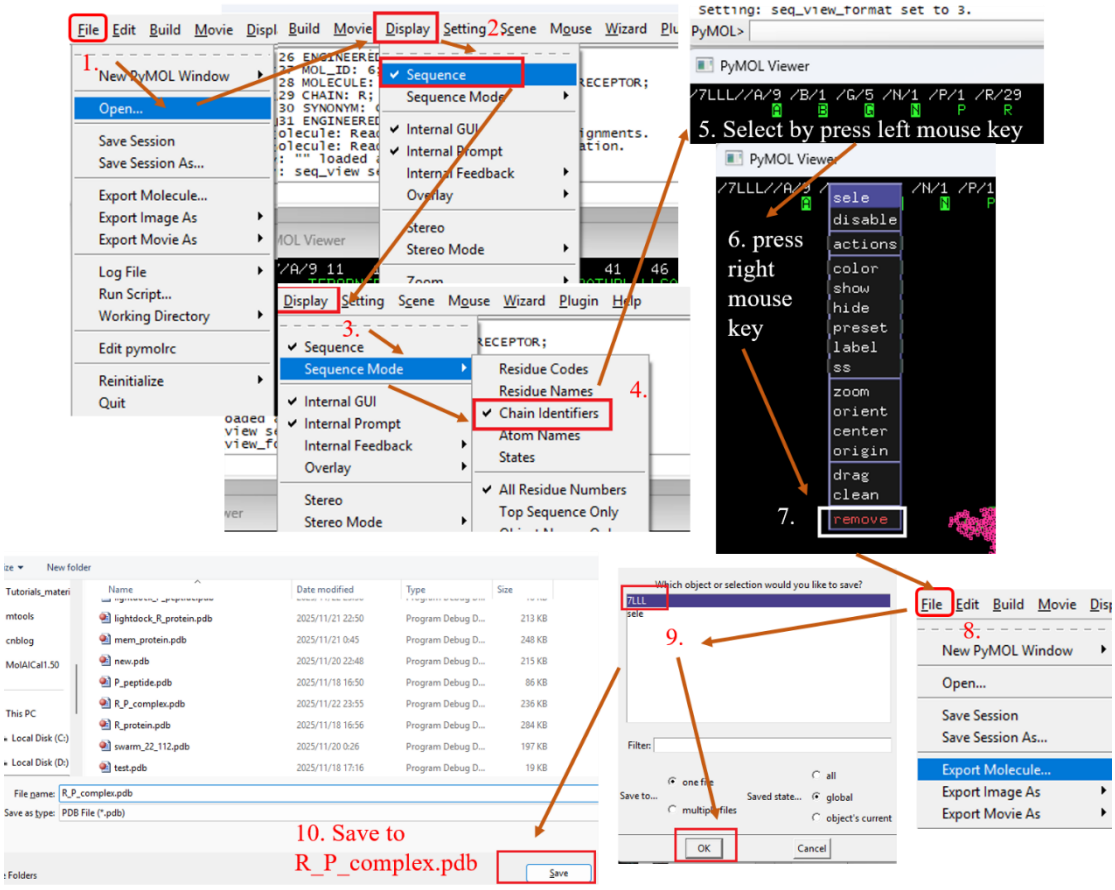

Figure 1

The agonist exendin-4 is a peptide. If users want to **perform protein-protein docking**, please **substitute the peptide file with the protein file** because the peptide can be considered a small protein.

Options: Sometimes, the chains of two different molecules are given the same name (e.g., both named Chain A). In such cases, users can rename the two molecules separately (e.g., as Chain A and Chain B) by calling VMD through MolAICal. **If this situation does not occur, please skip** this step and the following commands. The command is as follows:

```
#> molaical.exe -call run -c vmdargs -i -pdb R_protein.pdb -args none set sel [atomselect top all], \\\$sel set chain A, \\\$sel writepdb R_protein_A.pdb
```

```
#> molaical.exe -call run -c vmdargs -i -pdb P_peptide.pdb -args none set sel [atomselect top all], \\\$sel set chain B, \\\$sel writepdb P_peptide_B.pdb
```

Note:

- It includes **all command-line arguments** of **external programs**, but the **'-i'** parameter must be the last one specified, while all other identifiers (e.g., '-call', '-c', etc.) must be **before '-i'.**
- **'-pdb'** means to load pdb file into VMD.
- **-c:** It will run the VMD command once in the Tk Console if its value is "vmdargs".

- The Strings after **'-args'** are represented to the commands which can be run on the Tk console of VMD.
  - The first parameter after **'-args'** can be a package name of VMD or **'none'**. If the first parameter is **'none'** after **'-args'**, it will omit the package for loading. For example, it will run the command "package require **autopsf**" in the Tk console of VMD if the first parameter is **'autopsf'** after **'-args'**. It will not run the command "package" if the first parameter is **'none'** after **'-args'**.
  - The string following **'-args'** contains one command that can be executed in VMD's Tk console before each comma **','**; in other words, the comma character replaces the process of entering each command in VMD's Tk console and pressing the Enter key. This allows multiple lines of commands to be executed with just a single command.
  - Some special characters, such as the character **"$"**, require the use of the escape character **"\"**. For special characters, MolAICal uses three escape characters **"\"** (i.e., **"\\\"**).

Option: The DFIRE scoring function in LightDock requires standard amino acid residues and their atom names; thus, it is essential to inspect and repair the receptor or ligand before performing docking with the DFIRE scoring function. The repair command is as follows:

```
#> molaical.exe -call run -c sfile -i lmfix_rec.py -i protein.pdb -o protein_fixed.pdb
```

Here, the PDB file is OK and does not need to be dealt with the above option.

In this tutorial, the above option is not used; the files "R_protein.pdb" (chain name is "R") and "P_peptide.pdb" (chain name is "P") are used because they have different chain names.

3.2. Make residue restraint file

Creating a restraint file for residues is like identifying the key residue sites in the active pocket, around which the molecule will be docked. **Users can customize these key residues based on literature introductions or directly from experience**.

The residue representation format in the restraint file is:

```
Chain.Residue_Name.Residue_Number[.Residue_Insertion]
```

Note: Here, Residue_Insertion is an optional single-character identifier appended to the standard residue number, commonly used in PDB files to distinguish inserted residues without renumbering the entire sequence. In bioinformatics and structural biology, insertion codes are critical for accurately tracking and describing specific protein residues, particularly when dealing with sequence variations or comparisons.

When Residue_Insertion is omitted, the general format is:

```
Chain.Residue_Name.Residue_Number
```

Or

```
Chain.Residue_Name.Residue_Number P
```

Note: The label "P" indicates that this restraint is passive (as opposed to active restraints, which have no label).

Passive residue restraints are not recommended unless their implications are fully understood.

Here, MolAICal is used to generate the residues list file (named 'rec.dat') of the receptor (protein) within 4.0 Å of the ligand (peptide) as key residues, as shown below:

```
#> molaical.exe -call run -c sfile -i get_res.tcl -pdb 7LLL.pdb -args R P 4.0 rec.dat R
```

Note: the terms 1st, 2nd, 3rd, and 4th below represent the first argument, the second argument, the third argument, and the fourth argument, respectively, and so on.

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **-c:** It will run the VMD scripts of MolAICal if its value is "sfile", here it calls the script file 'get_res.tcl'.
- **'-pdb'** means to load pdb file into VMD.
- **1st:** The chain name of the selected molecule;
- **2nd:** The chain name of reference molecule used for selection;
- **3rd:** The residues of the selected molecule within the specified distance from the reference molecule will be selected;
- **4th:** The saved filename;
- **5th:** Specifies the molecular type, 'R' stands for Receptor and 'L' stands for Ligand.

Similarly, MolAICal is used to generate the residues list file (named 'lig.dat') of the ligand (peptide) within 4.0 Å of the receptor (protein) as key residues, as shown below:

```
#> molaical.exe -call run -c sfile -i get_res.tcl -pdb 7LLL.pdb -args P R 4.0 lig.dat L
```

Merge the 'rec.dat' and 'lig.dat' into the restraint file named 'restraints.list' as follows:

```
#> molaical.exe -call run -c ecommand -i cat rec.dat lig.dat > restraints.list
```

In many cases, the specific details of the ligand are unknown. Therefore, this tutorial only uses the key residues of the receptor (the first molecule) for constraints (and manually removes unnecessary amino acids based on experience), with the commands as follows:

```
#> molaical.exe -call run -c ecommand -i cp rec.dat restraints.list
```

3.3. Setup for molecular docking

Firstly, run the setup for swarm generation as follows:

```
#> molaical.exe -call run -c ldock -i lightdock3_setup.py R_protein.pdb P_peptide.pdb --noxt --noh --now -rst restraints.list -sp
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.

- **-c:** It will run molecular docking (protein-protein, protein-peptide, protein-DNA or protein-RNA) by calling "LightDock" if its value is "ldock".
- The 1st and 2nd arguments after "lightdock3_setup.py" are receptor and ligand files, respectively.
- **--noxt**: If this option is enabled, LightDock ignores OXT atoms. This is useful for several scoring functions that don't understand this special type of atom.
- **--noh**: If this option is enabled, LightDock ignores hydrogen atoms. This is relevant for dfire, fastdfire or dfire2 scoring functions (among others). Only removes atoms starting with H character, thus not recognizing types such as 1H, etc.
- **--now**: If this option is enabled, crystal waters will be removed.
- **--anm:** If this option is enabled, the ANM mode is activated and backbone flexibility is modeled using ANM (via ProDy). **This parameter may cause structural distortion** in docking results by activating backbone flexibility modeled via ANM (using ProDy). **Avoid using it unless implementing better conformational sampling (e.g., energy minimization)**. Instead, pre-generate multiple conformations to bypass backbone flexibility during docking.
- **-membrane:** When enabled, this flag considers the receptor partner is aligned to the Z-axis and filters out swarms which would not be compatible with a trans-membrane domain. To do so, it makes use of the residues provided as receptor restraints.
- **-transmembrane:** The argument "--transmembrane" has the opposite function of "--membrane". When enabled, this flag considers the receptor partner to be aligned to the Z-axis and filters out swarms that would not be inside the membrane.
- **-sp:** If enabled (False by default), writes in the init folder debugging extra files.
- **-r, -rst restraints_file:** If restraints_file is provided, residue restraints will be considered during the setup and the simulation. If restraints_file is not provided, the setup and the simulation will focus on the entire receptor.

- ✧ It will produce the folder named 'init', which contains the molecules in PDB format that represent the glowworms of swarms for further docking.
- ✧ The files 'lightdock_R_protein.pdb' and 'lightdock_P_peptide.pdb' are re-generated from 'R_protein.pdb' and 'P_peptide.pdb' by LightDock

Open 'lightdock_R_protein.pdb' and all PDB files with the prefix "starting_positions*" in the folder named 'init' by PyMol (see Figure 2):

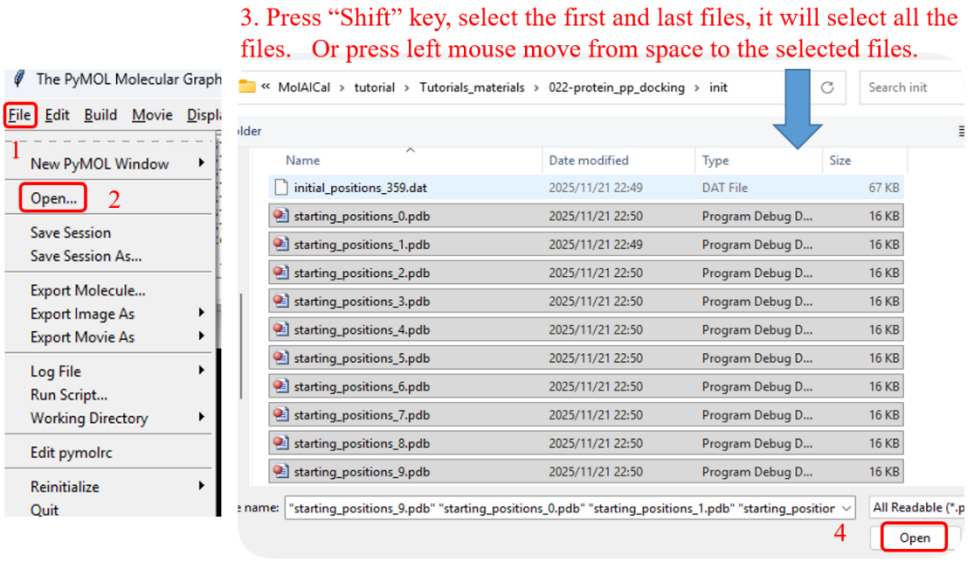

Figure 2

Please see the results in Figure 3. It is evident that many glowworms are **located in the intramembrane region**, while the ligand peptides are positioned in the extracellular-facing pocket at the center of the 7-transmembrane helices of the membrane protein (as shown in Figure 3). **These glowworms in the intramembrane region are not only unreasonable but also consume significant computational resources and time during calculations**, potentially compromising docking results.

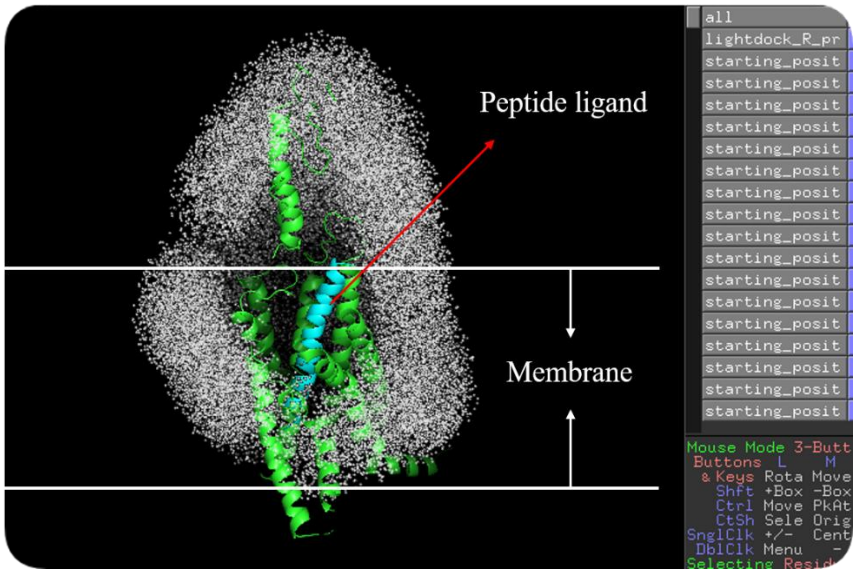

Figure 3

Therefore, MolAICal offers a simple membrane generation method to minimize the generation of unreasonable intramembrane glowworms:

3.3.1 (Option) Set the membrane in the receptor

Skip this step if the target receptor is not a membrane protein!

3.3.1.1 (Option) First, open "lightdock_R_protein.pdb" using VMD. Through VMD, it can visually inspect the receptor in x, y, and z directions. To determine whether the transmembrane region of the membrane protein (facing the extracellular side) is roughly aligned along the z-axis, if it is already aligned, no further action is needed. If it is not roughly aligned along the z-axis, use MolAICal to adjust its orientation to align with the z-axis. The command is as follows:

```
#> molaical.exe -call run -c sfile -i align_axis.tcl -pdb R_protein.pdb -args protein 2 new.pdb
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **-c:** It will run the VMD scripts of MolAICal if its value is "sfile", here it calls the script file 'align_axis.tcl'.
- **'-pdb'** means to load pdb file into VMD.
- **1st:** Atom selection (same as the command "atomselect" in the Tk console of VMD). If there is one more than a letter (e.g., protein and chain B), it will use the comma "," to represent the blank space " " (e.g., protein,and,chain,B);
- **2nd:** If the protein aligns horizontally (lying flat), change 0 to 2;
- **3rd:** The output file name;
- **4th:** The selected axis for "transaxis" of VMD, which can be x, y, z, or "". "" can be equal to no character, including space for inputting.

The file "new.pdb" is along the z-axis, which will be used for the steps below.

3.3.1.2 Once, make sure the receptor is roughly along the z-axis. Identify two positions on the receptor for membrane generation.

- Firstly, open "R_protein.pdb" in VMD
- Secondly, hold down the numeric key '1', then roughly click with the mouse near the inner and outer edges of the membrane. These positions can be adjusted according to experimental requirements, primarily to restrict the generation of glowworms between the membranes.
- Obtain the z-axis values of these two positions and use them as the min and max values of the range [min, max] (as shown in Figure 4).

Here, [min, max]=[-29.0, 10.0]. These are just the rough numbers. Users can adjust them according to the research purpose. Then, run the command below for membrane generation:

```
#> molaical.exe -call run -c sfile -i gen_mem.tcl -pdb new.pdb -args protein -29.0 10.0 mempro.pdb
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **-c:** It will run the VMD scripts of MolAICal if its value is "sfile", here it calls the script file 'gen_mem.tcl'.
- **'-pdb'** means to load pdb file into VMD.
- **1st:** Atom selection (same as the command "atomselect" in the Tk console of VMD). If there is one more than a letter (e.g., protein and chain B), it will use the comma "," to represent the blank space " " (e.g., protein,and,chain,B);
- **2nd:** The minimum z-axis for membrane generation;

- **3rd:** The maximum z-axis for membrane generation;
- **4th:** The saved output file name;
- **5th:** The delta value is used to [($max - $delta), $max], which represents the range of values used to determine the maximum and minimum x,y values along the z-axis region. This ensures that membrane particles in the cross-section of the membrane are not located inside the membrane protein. Default value: 3.0;
- **6th:** Safety buffer for the bounding box (set 0 to strictly use protein limits). Default value: -2.0;
- **7th:** Membrane Padding (Angstroms). Default value: 15.0;
- **8th:** The minimum spacing between particles. Default value: 3.5;
- **9th:** The maximum spacing between particles. Default value: 6.0;
- **10th:** The distance to keep away from protein atoms (Angstroms). Default value: 4.0.

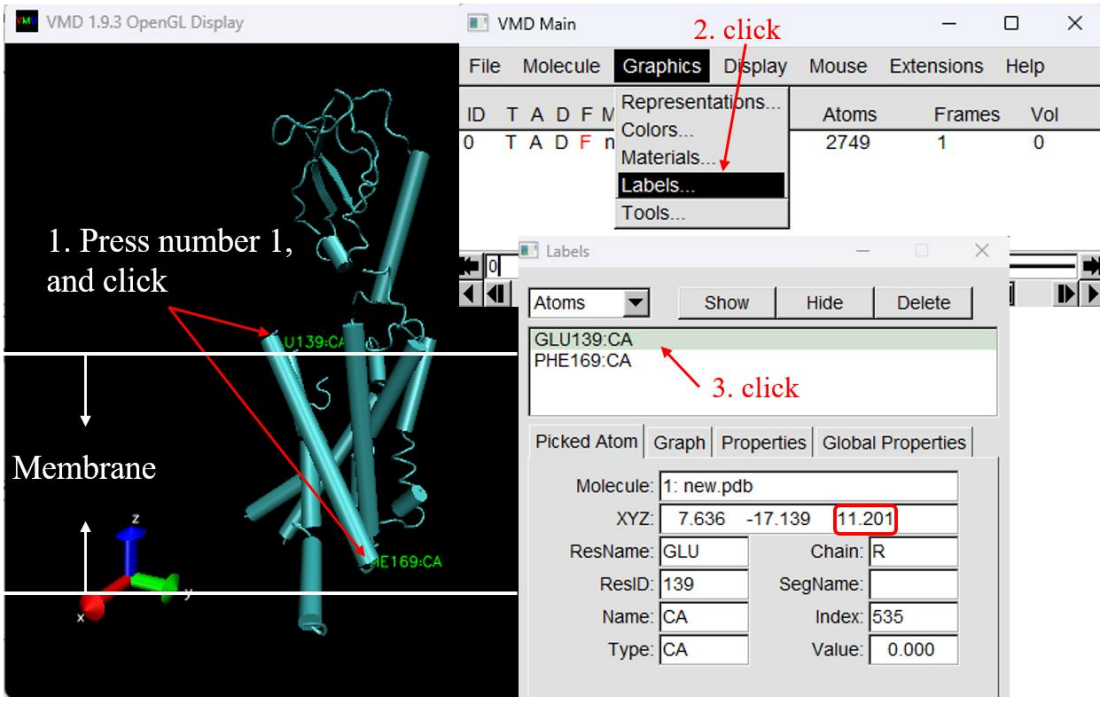

Figure 4

3.3.1.3 It will generate the file named "mempro.pdb", which contains receptor and membrane molecules.

- **Option:** Remove the previously generated files (If not deleted, it will cause some errors). If the previously generated files do not exist, skip this step.

```
#> molaical.exe molaical.exe -call run -c ecommand -i rm -rf init swarm_* lightdock_*
```

Re-run the setup for swarm generation as follows (add argument: --membrane):

```
#> molaical.exe -call run -c ldock -i lightdock3_setup.py mempro.pdb P_peptide.pdb -membrane --noxt --noh --now -sp -rst restraints.list
```

Notice: -transmembrane: The argument **"--transmembrane"** has the opposite function of **"--membrane".** When enabled, this flag considers the receptor partner to be aligned to the Z-axis and filters out swarms that would not be inside the membrane.

It is obvious that the number of generated swarms is smaller than before. Open '**lightdock_mempro.pdb**' and all **PDB** files with the **prefix** "starting_positions*" in the folder named **'init'** by PyMol as shown in Figure 2. **Regenerate** glowworms of swarms as shown in **Figure 5;** notably, no swarms are generated between membranes, which is reasonable.

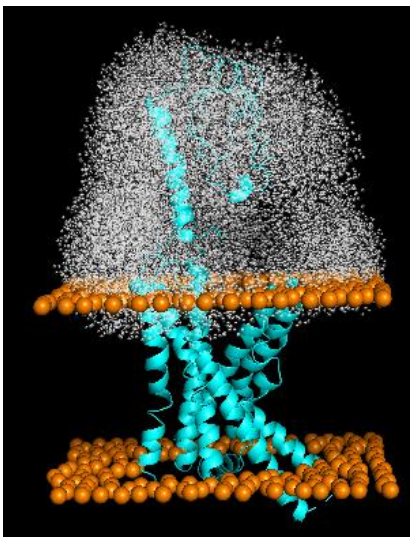

Figure 5

3.4. Simulation for molecular docking

Secondly, run the simulation:

```
#> molaical.exe -call run -c ldock -i lightdock3.py setup.json 100 -s fastdfire -c 12 -min
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **-c:** It will run molecular docking (protein-protein, protein-peptide, protein-DNA or protein-RNA) by calling "LightDock" if its value is "ldock".
- The 1st and 2nd arguments after "lightdock3.py" are the configuration file generated on the setup step, and the number of steps of the simulation (here, it is 100 steps), respectively.
- **-s SCORING_FUNCTION:** The default scoring function (DFIRE, fast C implementation fastdfire) using this flag. A name of a scoring function or a file containing the name and weight of multiple scoring functions are accepted.
- **-c CORES:** By default, the total number of available CPU cores on the hardware is used to run the simulation, but a different number of CPU cores can be specified via this option.
- **-min:** When enabled, the **'-min'** option performs local minimization of the top-scoring particle per swarm step using SciPy's scipy.optimize.fmin_powell algorithm.

3.5. Generate models, Clustering, Rank, and filter

After the simulation is completed, MolAICal calls the script to generate models, cluster, rank, and

filter the results as follows:

```
#> molaical.exe -call run -c sfile -i lrank.sh 1::=mempro.pdb 2::=P_peptide.pdb 3::=R 4::=P 5::=15 6::=restraints.list 13::=0.7
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **'::= '**: Here, it is used to assign values to parameters in the specified order.
- **'-c'**: It will run the VMD scripts of MolAICal if its value is "sfile".
- **'1::= '**: It is the path of the first input molecule (receptor). Required parameters.
- **'2::= '**: It is the path of the second input molecule (ligand). Required parameters.
- **'3::= '**: Chains on the receptor partner (1st molecule). The default value is 'R'.
- **'4::= '**: Chains on the ligand partner (2nd molecule). The default value is 'P'.
- **'5::= '**: The number of used CPU cores. The default value is **12**.
- **'6::= '**: File including restraints. The default value is 'restraints.list'.
- **'13::= '**: Structures with at least this fraction of native contacts for the filter step. The default value is **0.4**. To identify ligands that bind to more key residues in the receptor's active site, the constraint ratio is set to 0.7 here.
- Other parameters use the default values. For more information, please see the MolAICal manual.

After the clustering and filtering processes are complete, a new directory named **"filtered"** is generated. This directory holds all predicted structures that meet the default 70% filtering criteria. Within it, a file named "rank_filtered.list" will be found, which ranks these structures based on the LightDock DFIRE (fastdfire) scores, where higher (more positive) scores indicate better rankings.

Table 1

Name	Score	Percentages
swarm_75_173.pdb	31.008	0.714
swarm_60_55.pdb	30.682	0.786
s_swarm_22_3.pdb	25.149	0.400
swarm_97_172.pdb	19.769	0.786

Note: Percentages corresponding to >13::=0.7 is calculated by len(contacts_receptor & restraints_receptor) + len(contacts_ligand & restraints_ligand) / (len(restraints_receptor) + len(restraints_ligand)). The molecular naming format: swarm_{id_swarm}_{id_glowworm}.pdb, where 'id_swarm' corresponds to the swarm folder name generated during 'setup', and 'id_glowworm' refers to the pdb-formatted molecule name within the swarm folder.

Load the "R_P_complex.pdb" which is the native binding conformation and the molecules in Table 1 to **PyMol**, align the "R_P_complex.pdb" to the wanted molecule (see Figure 6). The docking pose clearly recapitulates structural similarity to the reference structure ("R_P_complex.pdb").

Table 1 shows the scores of the docked results. The "s_swarm_22_3.pdb" candidate compound in Table 1, generated through variation of membrane insertion depths, serves as a case for comparative analysis in this tutorial.

Besides, users can put the best one or the assigned top molecules to a new folder

"**filtered**/candidates" as command below:

```
#> molaical.exe -call run -c sfile -i mrank.py -ft 1
```

➢ -t: Number of top molecules to copy to candidates (default: 0)

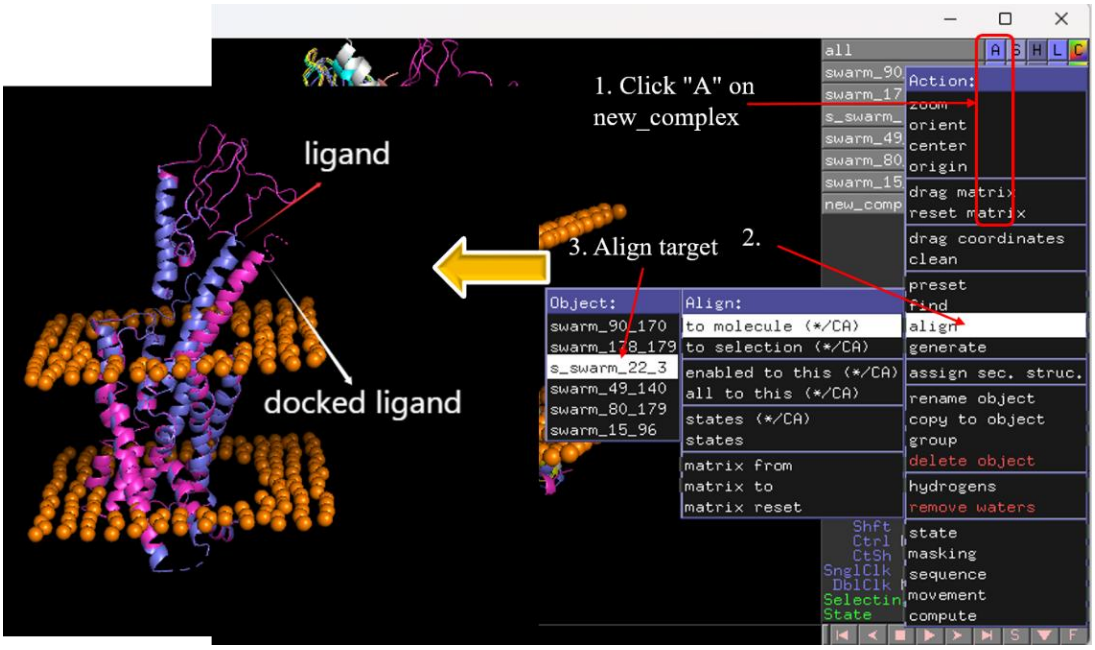

Figure 6

3.6. MM/GBSA calculation for further optimization

MolAICal computes MM/GBSA and MM/PBSA binding free energies. Based on literature evidence and empirical validation, MM/GBSA demonstrates superior performance over MM/PBSA for protein-protein and protein-ligand docking systems. However, this does not imply that MM/GBSA universally outperforms MM/PBSA across all systems. We recommend that researchers select the optimal method based on their specific biomolecular system and computational objectives. Here, the workflow employs MM/GBSA to calculate binding affinities for the docking poses, with the native complex structure 'R_P_complex.pdb' serving as the benchmark reference in comparative MM/GBSA calculations.

```
#> molaical.exe -call run -c sfile -i mmgbpbsa_batch.sh 1::=R 2::=P
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **'::= '**: Here, it is used to assign values to parameters in the specified order.
- **'-c'**: It will run the VMD scripts of MolAICal if its value is "sfile".
- **'1::= '**: Chain of the first molecule. The default value is 'R'.

- **'2::= '**: Chain of the second molecule. The default value is 'P'.
- Other parameters use the default values. For more information, please see the MolAICal manual.

Notice:

1) During initial execution, a parameter file named **"cal_mmgbpbsa.sh"** is automatically generated. Subsequent runs will not overwrite an existing **"cal_mmgbpbsa.sh"** file; the current version in the directory takes precedence. Users may customize parameters within **"cal_mmgbpbsa.sh"**, which **adheres** to **MolAICal's script format** enabling **batch execution** of MolAICal commands line-by-line (command-line calls typically begin with molaical.exe).

2) MolAICal currently only supports automatic calculation of MM/GBSA and MM/PBSA for standard protein residues and nucleic acids. For small molecules and non-standard amino acids, additional CHARMM force fields are required, and the script named "cal_mmgbpbsa.sh" in the directory can be modified by adding parameters such as "-top" and "-ff" for adding the additional CHARMM force fields. Specifically, changing the 5th line of "cal_mmgbpbsa.sh" switches the calculation to MM/PBSA. Detailed information can be found in the MolAICal manual and corresponding MM/GBSA and MM/PBSA tutorials.

Ranking or extracting the wanted molecules to the './mmgbpbsa /candidates' as the commands below:

1. Only print the rank results of MM/GB/PBSA, run command below:

```
#>molaical.exe -call run -c sfile -i mrank.py
```

2. Copy the top 2 molecules of the results to the **'./mmgbpbsa /candidates', run command below:**

```
#> molaical.exe -call run -c sfile -i mrank.py -t 2
```

➢ -t: Number of top molecules to copy to candidates (default: 0)

It will show the results as below:

Name	G_binding (kcal/mol)	±	SE	SD
R_P_complex.pdb	-48.4655	+/-	0.0140	0.0989
swarm_97_172.pdb	-37.9548	+/-	0.0408	0.2884
swarm_60_55.pdb	-37.1118	+/-	0.0227	0.1605
swarm_75_173.pdb	-36.8318	+/-	0.0047	0.0334
s_swarm_22_3.pdb	-11.4960	+/-	0.0164	0.1163

Under normal circumstances, the more negative the G_binding value, the better. The "R_P_complex.pdb" is native conformation. It indicates MM/GBSA is a good method to assess the binding affinity between two molecules.

Notice: This tutorial provides detailed, **multi-step instructions primarily for teaching purposes**.

While the **steps** in this tutorial have been **updated**, the tutorial **results** have **not** been **refreshed**; if your output differs from the tutorial's, trust your actual results.

If the program is terminated abnormally (**e.g., via "Ctrl+C" or other non-standard methods**), **numerous residual processes may continue running uncontrollably.** To resolve this issue, users can delete all processes associated with the current directory using the following command::

```
#> molaical.exe -eset pdclean
```

Note:

- Execute this command multiple times to ensure complete removal of related processes.
- WARNING: This will also terminate processes of OTHER RUNNING PROGRAMS in the same directory.
- Verify carefully before execution:
  - If multiple programs are running from this directory, explicitly confirm which directory-associated processes are safe to delete.
  - Never run this command in shared/production directories without explicit validation."

References

1. Jiménez-García, B.; Roel-Touris, J.; Romero-Durana, M.; Vidal, M.; Jiménez-González, D.; Fernández-Recio, J., LightDock: a new multi-scale approach to protein-protein docking. *Bioinformatics* **2018,** *34* (1), 49-55.
2. Krishnanand, K. N.; Ghose, D., Glowworm swarm optimization for simultaneous capture of multiple local optima of multimodal functions. *Swarm Intelligence* **2009,** *3* (2), 87-124.

Appendix 1

Install external programs inside the MolAICal container (if finished, only for Linux Version MolAICal)

Please note that the configuration of MolAICal differs between the Windows and Linux versions: the Linux version utilizes the udocker container approach and requires setup within the container, whereas the Windows version does not.

To address the library dependency issues in Linux, the Linux version of MolAICal adopts a container-based approach. If **external programs need** to be invoked, it is **recommended** to install them **within** the MolAICal **container**. If **no external** programs are required, this step can be **ignored**. This tutorial requires the external programs NAMD and VMD, and the steps are as follows:

a) First of all, copy the files to the MolAICal container.

```
****** 1.  Go to the container file system, it will enter the "/root" ******
#> molaical.exe -eset shell in

****** 2.  The first part of 'cp' is from the local machine, and the second part is in the container; ******
****** The VMD and NAMD packages can be moved into the container by the command below ******
#> cp /home/user/<local machine file> /root/soft

****** 3.  Exit container file system ******
#> exit
```

b) Secondly, enter the container virtual environment of MolAICal:

```
#> molaical.exe -eset sys run molaical
```

Note: 'molaical' is the container name.

c) Installing software in the container virtual environment of MolAICal is the same as installing it on the host local machine. Here, we take the installation of VMD and NAMD as examples:

1) For NAMD:

Extract the NAMD files, assuming the extracted folder is named namdcpu. Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter

appears to be ineffective for reproducibility in the CUDA version of NAMD.

2) For VMD:

Following the steps below:

✧ Extract the VMD files:

```
#> tar -xzvf vmd-xxx.tar.gz
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system.

✧ Modify the install path in a file named "**configure**" in the decompressed folder of VMD:

```
The default: $install_bin_dir="/usr/local/bin"; modify it to →
$install_bin_dir="/root/soft/vmd193/bin";

The default: $install_library_dir="/usr/local/lib/$install_name"; modify it to →
$install_library_dir="/root/soft/vmd193/lib/$install_name";
```

✧ Install VMD:

```
#> cd vmd-xx
#> ./configure LINUXAMD64
#> cd src
#> make install
```

Note: Please run **./configure**, and select the right types for the used computers. Here, it is "LINUXAMD64".

✧ Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
```

So far, all the installations and configurations of NAMD and VMD are complete in the MolAICal container.

✧ **Remember to use the "exit" command to quit the MolAICal virtual environment and return to the local computer for computation (primarily to avoid file-copying steps; execution within the container is also possible but requires copying data from the local computer to the MolAICal container).**

```
#> exit
```

3) Save and load the modified container (Option)

To preserve changes made within a container and avoid reinstalling software later, since all data will be lost if the container is deleted.

◆ The first step is to obtain the container ID and name:

```
#> molaical.exe -eset sys ps
```

◆ Secondly, clone this container as follows:

```
#> molaical.exe -eset sys export --clone -o molaicalv2.tar molaical
```

Note: 'molaical' is the container name. It can also be the container ID. If an error is reported, it may

be due to permission issues with the mounted file system, which can be ignored.

- Thirdly, import the cloned container **if needing**:

```
#> molaical.exe -eset sys import --clone --name molaicalv2 molaicalv2.tar
```

- Now, change the loaded new container name to the default container name "molaical", the previous container is renamed to "bakmolaical", the commands are as follows:

```
#> molaical.exe -eset sys rename molaical bakmolaical
#>molaical.exe -eset sys rename molaicalv2 molaical
```

Notice: Please note that a **container** (**dynamic**) is generated from an **image** (**static**). Operations such as software installation must be performed within the dynamic container; if cloning this container, restoring it will still be based on the original underlying image.
For more information, please see the manual of MolAICal or run "molaical.exe -eset sys --help" to get more help details.